Andy Zou, Long Phan, Justin Wang, Derek Duenas, Maxwell Lin, Maksym Andriushchenko, Rowan Wang, Zico Kolter, Matt Fredrikson, and Dan Hendrycks. Improving Alignment and Robustness with Circuit Breakers. *arXiv*, June 2024. doi: 10.48550/arXiv.2406.04313. URL https://arxiv.org/abs/2406.04313v4.

Jakub Łucki, Boyi Wei, Yangsibo Huang, Peter Henderson, Florian Tramèr, and Javier Rando. An Adversarial Perspective on Machine Unlearning for AI Safety. *arXiv*, 2024. doi: 10.48550/arXiv.2409.18025. URL https://arxiv.org/abs/2409.18025.

A Details on harmfulness detection

A.1 Harmfulness Monitors

Dataset collection. For our benign dataset $\mathcal{D}_{\text{benign}}$, we collate inputs from UltraChat [Ding et al., 2023], a large-scale dialogue dataset, and Xstest [Röttger et al., 2023], benign queries that are often mistakenly refused by language models. For each benign query, we sample a response using Llama-3-8B-Instruct. We filter the benign input dataset to only include inputs that do not lead to refusal from the model (finding that we only need to remove a very small number of inputs). As our harmful $\mathcal{D}_{\text{harm}}$ dataset, we use the Circuit Breakers dataset [Zou et al., 2024] for both inputs and generations. We train probes on training splits of these datasets as described in Eq. (4).

From $\mathcal{D}_{\text{harm}}$ we derive different datasets for several prominent jailbreaking methods, such as GCG [Zou et al., 2023b], PAIR [Chao et al., 2023], MSJ [Anil et al., 2024], and AutoDAN [Liu et al., 2023], among others (see Appendix A.2). Each method modifies a harmful request to make the model follow the request. If the modified query leads to Llama-3-8B-Instruct generating a helpful and harmful response (according to the StrongReject jailbreaking classifier [Souly et al., 2024]), then we add the modified harmful query and resulting harmful generation to a new dataset. These jailbreak specific datasets are used to evaluate monitors' ability to detect jailbroken inputs / outputs in §3.2.

Monitor training. Using the training subset of $\mathcal{D}_{\text{harm}}$ we train linear, mean cosine difference, MLP probes, and Gaussian OOD detectors on Llama-3-8B-Instruct. Our MLP probe contains a single hidden layer of size 64. For the MLP and linear probes, we train using the Adam optimizer with 10^{-3} learning rate, and 10^{-5} weight decay, and batch size 64. For mean cosine difference probes we simply calculate the mean of the harmful and benign activation clusters and take the difference. At test time, we return the average cosine similarity between the activations at each layer and the mean difference direction at each layer. For the Gaussian OOD detector, we fit a separate mean and covariance matrix to the benign activations of $\mathcal{D}_{\text{benign}}$ for each layer of the model, using a weighted average of the standard empirical estimates and a diagonal regularization matrix.

As mentioned in §2.1, we train input and generation monitors. Formally, for parameters θ_{input} and θ_{gen} of input and output based monitors D_θ respectively, we solve the optimization problem in Eq. (4).

$$\begin{aligned}\theta^*_{\text{input}} &= \arg\min_{\theta} \sum_{\texttt{inp},\texttt{out},y} \text{CrossEntropy}(D_\theta(\mathcal{R}(\texttt{inp})[-1]), y) \\ \theta^*_{\text{gen}} &= \arg\min_{\theta} \sum_{\texttt{inp},\texttt{out},y} \text{CrossEntropy}(D_\theta(\mathcal{R}(\texttt{inp} \oplus \texttt{out})[\texttt{len(inp):}]), y),\end{aligned} \tag{4}$$

where $\oplus$ refers to string concatenation, `inp`, `out` are input and output text that is harmful or not according to the binary label y. We refer to the resulting probes as D_{input} and D_{gen}.

For each of these probes, we train one embedding suffix with the training split of $\mathcal{D}_{\text{harm}}$, and test this suffix using the testing set of $\mathcal{D}_{\text{harm}}$. For hard prompts, we select a subset of 20 harmful queries from $\mathcal{D}_{\text{harm}}$ and for each prompt and probe, train a hard prompt suffix using the FLRT optimizer. We show the performance of the probes and obfuscation attacks in Fig. 4b.

Linear ensemble probe. In §3.5, we introduce a new *Ensemble* probe. We explain the details of this probe here. Put simply, it is an ensemble of mean difference probes. Recall that §3.5 concerns

	Probe	GCG	PAIR	AutoDAN	MSJ	Multi-Turn	Prefill	Misc.	Average
Instruction	Mean Diff	16.48	0.38	0.00	100.00	32.99	98.98	28.75	39.66
	LogReg	92.31	64.02	100.00	100.00	76.29	100.00	58.75	84.48
	Quadratic	87.91	94.32	100.00	100.00	86.94	100.00	71.25	91.49
	MLP	95.60	83.33	100.00	100.00	87.29	100.00	66.25	90.35
Generation	Mean Diff	63.74	23.11	95.90	100.00	53.95	91.88	75.00	71.94
	LogReg	95.60	95.45	100.00	100.00	89.69	99.49	87.50	95.39
	Quadratic	95.60	96.21	100.00	100.00	90.03	99.49	90.00	95.91
	MLP	96.70	96.59	100.00	100.00	90.72	100.00	90.00	96.29

Table 1: Jailbreak Detection Performance for Different Probes (Recall @ 1% FPR)

an iterated adversarial training game in which the defender trains against obfuscated activations and the attacker attempts to find new obfuscated activations. Each time the defender goes, they calculate the mean difference between the set of benign activations and harmful obfuscated activations induced by the most recent attack embedding. They use this direction to construct a new mean difference probe, and add it to the running ensemble. To predict using this ensemble, they simple take the max prediction from each probe. Thus the prediction of the ensemble is simply the *maximum cosine similarity to any direction present in the ensemble*.

A.2 Details Regarding Standard Jailbreaks

In order to test the ability of latent defenses to safeguard against harmful completions, we evaluated our probes on their ability to correctly flag jailbreaks. Here, we expand on the various different jailbreak types that we used.

- **GCG** [Zou et al., 2023b] Optimizes a 15-token suffix appended to prompts using discrete optimization. Each attack is input-specific and maximizes likelihood of harmful completions.
- **PAIR** [Chao et al., 2023] Uses GPT-4 to iteratively rewrite harmful prompts until bypassing model refusal. Often generates jailbreaks disguised as creative writing scenarios.
- **AutoDAN** [Liu et al., 2023] Uses genetic algorithms to evolve jailbreak prompts from core templates, combining evolutionary search with prompt engineering.
- **Many-Shot Jailbreaking** [Anil et al., 2024] Exploits in-context learning using 80 examples of harmful requests paired with helpful responses to condition model behavior.
- **Multi-Turn Attacks** [Li et al., 2024, Haize, 2024] A combination of expert-crafted multi-turn conversations that result in harmful completions, and multi-turn conversations generated automatically by an attacker LLM to break a target model.
- **Prefill** Generates first 20 tokens using a helpful-only model, then lets target model continue. Exploits observation that models rarely refuse mid-completion.